Lecture 4. Iterations, Generators, and Programmer-Defined Datatypes

Programming Paradigms, Fall 2026 — 16 Sep 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. Iterations
3. Sequences and Streams
4. Generators
5. Programmer-Defined Datatypes

Where We Are

This Lecture

Last week: the recursion patterns over lists became functions.

This Lecture

Last week: the recursion patterns over lists became functions.

This week we look at functions that produce their result **incrementally**.
That is, the first results are ready before the whole input has been read.

This Lecture

Last week: the recursion patterns over lists became functions.

This week we look at functions that produce their result **incrementally**.
That is, the first results are ready before the whole input has been read.

Three things to get out of it:

1. **Writing** a loop as a `for` form, and **reading** one.
2. **Producing** values on demand, with streams and generators, and writing list functions in the same style.
3. **Defining** your own datatypes with `struct`, so that the values in a pipeline have names for their parts.

Where We Stopped

Recall the `foldl`, essentially a `foreach`-loop with local state:

```
(foldl (lambda (student current)
         (if (> (cdr student) 4.0)
             (cons (car student) current)
             current))
       empty students)
; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Where We Stopped

Recall the `foldl`, essentially a `foreach`-loop with local state:

```
(foldl (lambda (student current)
         (if (> (cdr student) 4.0)
             (cons (car student) current)
             current))
       empty students)
; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

1. Can we have the *syntax* of a loop as well?

Where We Stopped

Recall the `foldl`, essentially a `foreach`-loop with local state:

```
(foldl (lambda (student current)
         (if (> (cdr student) 4.0)
             (cons (car student) current)
             current))
       empty students)
; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

1. Can we have the *syntax* of a loop as well?
   Yes: the `for` forms, this hour.

Where We Stopped

Recall the `foldl`, essentially a `foreach`-loop with local state:

```
(foldl (lambda (student current)
         (if (> (cdr student) 4.0)
             (cons (car student) current)
             current))
       empty students)
; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

1. Can we have the *syntax* of a loop as well?
   Yes: the `for` forms, this hour.
2. Can we get results *incrementally*, before the whole list is done?

Where We Stopped

Recall the `foldl`, essentially a `foreach`-loop with local state:

```
(foldl (lambda (student current)
         (if (> (cdr student) 4.0)
             (cons (car student) current)
             current))
       empty students)
; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

1. Can we have the *syntax* of a loop as well?
   Yes: the `for` forms, this hour.
2. Can we get results *incrementally*, before the whole list is done?
   Yes: streams and generators, after the break.

Where You Have Met This Already

Consider the first three errors in a log file that is larger than memory, in four languages:

a shell

```
grep ERROR server.log | head -3
```

Python

```
from itertools import islice
errors = (l for l in open("server.log")
            if "ERROR" in l)
first3 = list(islice(errors, 3))
```

Java

```
Files.lines(path)
     .filter(l -> l.contains("ERROR"))
     .limit(3)
     .toList();
```

SQL

```
SELECT line FROM log
WHERE line LIKE 'ERROR%' LIMIT 3
```

Where You Have Met This Already

Consider the first three errors in a log file that is larger than memory, in four languages:

a shell

```
grep ERROR server.log | head -3
```

Python

```
from itertools import islice
errors = (l for l in open("server.log")
            if "ERROR" in l)
first3 = list(islice(errors, 3))
```

Java

```
Files.lines(path)
     .filter(l -> l.contains("ERROR"))
     .limit(3)
     .toList();
```

SQL

```
SELECT line FROM log
WHERE line LIKE 'ERROR%' LIMIT 3
```

Each reads one line at a time, keeps/drops it, and stops after the third match. The file is never fully in memory (with `tail -f` it does not even have to end).

Where You Have Met This Already

Consider the first three errors in a log file that is larger than memory, in four languages:

a shell

```
grep ERROR server.log | head -3
```

Python

```
from itertools import islice
errors = (l for l in open("server.log")
            if "ERROR" in l)
first3 = list(islice(errors, 3))
```

Java

```
Files.lines(path)
     .filter(l -> l.contains("ERROR"))
     .limit(3)
     .toList();
```

SQL

```
SELECT line FROM log
WHERE line LIKE 'ERROR%' LIMIT 3
```

Each reads one line at a time, keeps/drops it, and stops after the third match. The file is never fully in memory (with `tail -f` it does not even have to end).

Note that `sort`, `wc -l`, `SUM` need the whole input.

The Principle: Incremental Results

The incremental test

Would the function still work if its input **may have no end**?

The Principle: Incremental Results



The incremental test

> Would the function still work if its input **may have no end**?

For example, `map` and `filter` pass this test, and so do `grep` and `head`.
Their first element depends only on the first elements of the input.

The incremental test

> Would the function still work if its input **may have no end**?

For example, `map` and `filter` pass this test, and so do `grep` and `head`.
Their first element depends only on the first elements of the input.

However, `reverse`, `sort` and `foldl` cannot pass it.
Their first element depends on the last element of the input.

The incremental test

> Would the function still work if its input **may have no end**?

For example, `map` and `filter` pass this test, and so do `grep` and `head`.
Their first element depends only on the first elements of the input.

However, `reverse`, `sort` and `foldl` cannot pass it.
Their first element depends on the last element of the input.

In this lecture, we use the test as a rule of style.

- Such a function usually has no accumulator and no final `reverse`.
- The **`for`** forms help here (they write the walk for us).
- Streams make the test literal, since a stream may indeed have no end.

Iterations

Brief Overview

1. Racket has a family of **`for`** forms, one per shape of result:
   - **`for`**, for effects
   - **`for/list`**, for a list
   - **`for/and`**, **`for/or`**, **`for/sum`**, **`for/product`**
   - **`for/fold`**, for anything
   - **`for*`** and **`for*/list`**, for nested loops

Brief Overview

1. Racket has a family of **`for`** forms, one per shape of result:
   - **`for`**, for effects
   - **`for/list`**, for a list
   - **`for/and`**, **`for/or`**, **`for/sum`**, **`for/product`**
   - **`for/fold`**, for anything
   - **`for*`** and **`for*/list`**, for nested loops
2. `in-range` and `in-naturals` are sequences of numbers to iterate over.

Brief Overview

1. Racket has a family of **`for`** forms, one per shape of result:
   - **`for`**, for effects
   - **`for/list`**, for a list
   - **`for/and`**, **`for/or`**, **`for/sum`**, **`for/product`**
   - **`for/fold`**, for anything
   - **`for*`** and **`for*/list`**, for nested loops
2. `in-range` and `in-naturals` are sequences of numbers to iterate over.
3. Streams and generators produce values on demand.

Iterations: `for`

`for` runs its body once per element, for the *effect*:

```
(for ([student students])
  (displayln (car student)))

; Anna
; Boris
; Charles
; Daria
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for`

`for` runs its body once per element, for the *effect*:

```
(for ([student students])
  (displayln (car student)))

; Anna
; Boris
; Charles
; Daria
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The square brackets bind `student` to each element in turn.

Iterations: `for`

`for` runs its body once per element, for the *effect*:

```
(for ([student students])
  (displayln (car student)))

; Anna
; Boris
; Charles
; Daria
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The square brackets bind `student` to each element in turn.

The value of the whole **`for`** is *void*: the results of the body are discarded. Thus, **`for`** is essentially an imperative loop.

Iterations: `for` with `cond`

A **`cond`** in the body skips the elements we do not want:

```
(for ([student students])
  (cond
    [(> (cdr student) 4.0)
     (displayln (car student))]))

; Anna
; Charles
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for` with `cond`

A **`cond`** in the body skips the elements we do not want:

```
(for ([student students])
  (cond
    [(> (cdr student) 4.0)
     (displayln (car student))]))

; Anna
; Charles
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

A **`cond`** with no matching clause does nothing, which is what we want here.

Iterations: `for/list`

`for/list` collects the body's results into a list, like `map`:

```
(for/list ([student students])
  (car student))

; '("Anna" "Boris" "Charles" "Daria")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/list`

`for/list` collects the body's results into a list, like `map`:

```
(for/list ([student students])
  (car student))

; '("Anna" "Boris" "Charles" "Daria")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Question

What does **`for/list`** give with the **`cond`** of the previous slide inside?

Iterations: `for/list` with `cond`

```
(for/list ([student students])
  (cond
    [(> (cdr student) 4.0)
     (car student)]))

; '("Anna" #<void> "Charles" #<void>)
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/list` with `cond`

```
(for/list ([student students])
  (cond
    [(> (cdr student) 4.0)
     (car student)]))

; '("Anna" #<void> "Charles" #<void>)
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Note that the body runs for *every* element, and a **`cond`** with no matching clause evaluates to void.

Iterations: `for/list` with `cond`

```
(for/list ([student students])
  (cond
    [(> (cdr student) 4.0)
     (car student)]))

; '("Anna" #<void> "Charles" #<void>)
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Note that the body runs for *every* element, and a **`cond`** with no matching clause evaluates to void.

A `cond` does not filter

A skipped element leaves nothing behind in **`for`**, and a hole in **`for/list`**.

Iterations: `for/list` with `#:when`

A **`#:when`** clause filters *before* the body runs:

```
(for/list ([student students]
           #:when (> (cdr student) 4.0))
  (car student))

; '("Anna" "Charles")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/list` with `#:when`

A **`#:when`** clause filters *before* the body runs:

```
(for/list ([student students]
           #:when (> (cdr student) 4.0))
  (car student))

; '("Anna" "Charles")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The list is built front to back, in one pass.
There is no accumulator, and nothing is reversed at the end.

Iterations: `for/list` with `#:when`

A **`#:when`** clause filters *before* the body runs:

```
(for/list ([student students]
           #:when (> (cdr student) 4.0))
  (car student))

; '("Anna" "Charles")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The list is built front to back, in one pass.
There is no accumulator, and nothing is reversed at the end.

`#:unless` is the same clause with the test negated.

Parallel Iteration

Two bindings in one **`for/list`** advance *together*:

```
(for/list ([i '(1 2 3 4)]
           [student students])
  (cons i (car student)))

; '((1 . "Anna") (2 . "Boris")
;   (3 . "Charles") (4 . "Daria"))
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Parallel Iteration

Two bindings in one **`for/list`** advance *together*:

```
(for/list ([i '(1 2 3 4)]
           [student students])
  (cons i (car student)))

; '((1 . "Anna") (2 . "Boris")
;   (3 . "Charles") (4 . "Daria"))
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The loop stops when the *shortest* sequence ends.

Parallel Iteration

Two bindings in one **`for/list`** advance *together*:

```
(for/list ([i '(1 2 3 4)]
           [student students])
  (cons i (car student)))

; '((1 . "Anna") (2 . "Boris")
;   (3 . "Charles") (4 . "Daria"))
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The loop stops when the *shortest* sequence ends.

Question

How do we number a list of any length?

Sequences of Numbers: `in-range` and `in-naturals`

`in-range` is a sequence of numbers, with an end:

```
(for/list ([i (in-range 5)])      i)  ; '(0 1 2 3 4)
(for/list ([i (in-range 1 5)])    i)  ; '(1 2 3 4)
(for/list ([i (in-range 0 10 3)]) i)  ; '(0 3 6 9)
```

Sequences of Numbers: `in-range` and `in-naturals`



`in-range` is a sequence of numbers, with an end:

```
(for/list ([i (in-range 5)])      i)  ; '(0 1 2 3 4)
(for/list ([i (in-range 1 5)])    i)  ; '(1 2 3 4)
(for/list ([i (in-range 0 10 3)]) i)  ; '(0 3 6 9)
```

`in-naturals` is the sequence of all naturals from a given one, with no end:

```
(for/list ([i (in-naturals 1)]
           [student students])
  (cons i (car student)))
; '((1 . "Anna") (2 . "Boris") (3 . "Charles") (4 . "Daria"))
```

Sequences of Numbers: `in-range` and `in-naturals`



`in-range` is a sequence of numbers, with an end:

```
(for/list ([i (in-range 5)])      i)  ; '(0 1 2 3 4)
(for/list ([i (in-range 1 5)])    i)  ; '(1 2 3 4)
(for/list ([i (in-range 0 10 3)]) i)  ; '(0 3 6 9)
```

`in-naturals` is the sequence of all naturals from a given one, with no end:

```
(for/list ([i (in-naturals 1)]
           [student students])
  (cons i (car student)))
; '((1 . "Anna") (2 . "Boris") (3 . "Charles") (4 . "Daria"))
```

This is fine here, since the loop stops with `students`, the shorter sequence.

Exercise: `indices-when`

Exercise 4.1

Implement `indices-when`, which lists the indices of the elements that satisfy a given predicate.

```
(indices-when even? '(1 2 3 5 7 9 10))
; '(1 6)
```

Exercise: `indices-when`

Exercise 4.1

Implement `indices-when`, which lists the indices of the elements that satisfy a given predicate.

```
(indices-when even? '(1 2 3 5 7 9 10))
; '(1 6)
```

Hint

Iterate over the list and over `(in-naturals)` in parallel.
The predicate selects the indices, so it goes in a **`#:when`** clause.

Exercise: `indices-when` (Solution)



```
(define (indices-when p? xs)
  (for/list ([x xs]
             [i (in-naturals)]
             #:when (p? x))
    i))

(indices-when even? '(1 2 3 5 7 9 10))  ; '(1 6)
```

Exercise: `indices-when` (Solution)

```
(define (indices-when p? xs)
  (for/list ([x xs]
             [i (in-naturals)]
             #:when (p? x))
    i))

(indices-when even? '(1 2 3 5 7 9 10))  ; '(1 6)
```

We can read this definition as a sentence:

1. for each `x` in `xs`, with its index `i`,

Exercise: `indices-when` (Solution)

```
(define (indices-when p? xs)
  (for/list ([x xs]
             [i (in-naturals)]
             #:when (p? x))
    i))

(indices-when even? '(1 2 3 5 7 9 10))  ; '(1 6)
```

We can read this definition as a sentence:

1. for each `x` in `xs`, with its index `i`,
2. when `x` passes `p?`,

Exercise: `indices-when` (Solution)

```
(define (indices-when p? xs)
  (for/list ([x xs]
             [i (in-naturals)]
             #:when (p? x))
    i))

(indices-when even? '(1 2 3 5 7 9 10))  ; '(1 6)
```

We can read this definition as a sentence:

1. for each `x` in `xs`, with its index `i`,
2. when `x` passes `p?`,
3. keep `i`.

Exercise: `indices-when` (Solution)

```
(define (indices-when p? xs)
  (for/list ([x xs]
             [i (in-naturals)]
             #:when (p? x))
    i))

(indices-when even? '(1 2 3 5 7 9 10))  ; '(1 6)
```

We can read this definition as a sentence:

1. for each `x` in `xs`, with its index `i`,
2. when `x` passes `p?`,
3. keep `i`.

The clauses come first, then the body.

Iterations: `for/list` vs `for*/list`

- With **`for/list`**, two sequences are walked *in parallel*.
- With **`for*/list`**, the second is walked *for each* element of the first.

```
(for/list
    ([i '(1 2 3)]
     [j (in-range 1 4)])
  (list i j))

; '((1 1) (2 2) (3 3))
```

```
(for*/list
    ([i '(1 2 3)]
     [j (in-range 1 4)])
  (list i j))

; '((1 1) (1 2) (1 3)
;   (2 1) (2 2) (2 3)
;   (3 1) (3 2) (3 3))
```

Iterations: `for/list` vs `for*/list`

- With **`for/list`**, two sequences are walked *in parallel*.
- With **`for*/list`**, the second is walked *for each* element of the first.

```
(for/list
    ([i '(1 2 3)]
     [j (in-range 1 4)])
  (list i j))

; '((1 1) (2 2) (3 3))
```

```
(for*/list
    ([i '(1 2 3)]
     [j (in-range 1 4)])
  (list i j))

; '((1 1) (1 2) (1 3)
;   (2 1) (2 2) (2 3)
;   (3 1) (3 2) (3 3))
```

Intuitively, **`for*`** is a nested loop.
Every `for/...` form has a `for*/...` version.

Iterations: `for/and` and `for/or`

`for/and` and **`for/or`** combine the body's results with **`and`** or **`or`**:

```
(for/and ([student students])
  (> (cdr student) 4.0))
; #f

(for/or ([student students])
  (> (cdr student) 4.0))
; #t
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/and` and `for/or`

`for/and` and **`for/or`** combine the body's results with **`and`** or **`or`**:

```
(for/and ([student students])
  (> (cdr student) 4.0))
; #f

(for/or ([student students])
  (> (cdr student) 4.0))
; #t
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

They stop early, as `andmap` and `ormap` do:
`for/and` at the first false result, **`for/or`** at the first true one.

Iterations: `for/sum` and `for/product`

`for/sum` and **`for/product`** add and multiply the body's results:

```
(for/sum ([student students])
  (cdr student))
; 14.9

(for/product ([i (in-range 1 5)])
  i)
; 24
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/sum` and `for/product`

`for/sum` and **`for/product`** add and multiply the body's results:

```
(for/sum ([student students])
  (cdr student))
; 14.9

(for/product ([i (in-range 1 5)])
  i)
; 24
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Each form names the shape of its result (a list, a boolean, or a number).
The body is an ordinary expression in every one of them.

Iterations: `for/fold`

`for/fold` is `foldl` with the syntax of a loop:

```
(for/fold ([result empty])
          ([student students]
           #:when (> (cdr student) 4.0))
  (cons (car student) result))

; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/fold`



`for/fold` is `foldl` with the syntax of a loop:

```
(for/fold ([result empty])
          ([student students]
           #:when (> (cdr student) 4.0))
  (cons (car student) result))

; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The first brackets name the accumulator and its start.
The body computes its next value.

Iterations: `for/fold`

`for/fold` is `foldl` with the syntax of a loop:

```
(for/fold ([result empty])
          ([student students]
           #:when (> (cdr student) 4.0))
  (cons (car student) result))

; '("Charles" "Anna")
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

The first brackets name the accumulator and its start.
The body computes its next value.

This is the fold we stopped at, without the **`lambda`**.
We use **`for/fold`** for results that need the whole input, such as a total or a maximum.

Iterations: `for/fold` with Several Accumulators

`for/fold` may carry several accumulators, returned together with `values`:

```
(for/fold ([names empty]
           [count 0])
          ([student students]
           #:when (> (cdr student) 4.0))
  (values
    (cons (car student) names)
    (+ 1 count)))

; '("Charles" "Anna")
; 2
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Iterations: `for/fold` with Several Accumulators

`for/fold` may carry several accumulators, returned together with `values`:

```
(for/fold ([names empty]
           [count 0])
          ([student students]
           #:when (> (cdr student) 4.0))
  (values
    (cons (car student) names)
    (+ 1 count)))

; '("Charles" "Anna")
; 2
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

With `foldl`, the two would have to be packed into one pair.

The `for` Forms and the Functions They Replace

the form	the function
`(for/list ([x xs]) e)`	`(map (lambda (x) e) xs)`
`(for/list ([x xs] #:when p) x)`	`(filter (lambda (x) p) xs)`
`(for/or ([x xs]) p)`	`(ormap (lambda (x) p) xs)`
`(for/and ([x xs]) p)`	`(andmap (lambda (x) p) xs)`
`(for/sum ([x xs]) e)`	`(apply + (map (lambda (x) e) xs))`
`(for/fold ([a z]) ([x xs]) e)`	`(foldl (lambda (x a) e) z xs)`
`(for*/list ([x xs] [y ys]) e)`	a `map` inside a `map`, then `apply append`
`(for ([x xs]) e)`	no function: it is run for the effects

The `for` Forms and the Functions They Replace

the form	the function
`(for/list ([x xs]) e)`	`(map (lambda (x) e) xs)`
`(for/list ([x xs] #:when p) x)`	`(filter (lambda (x) p) xs)`
`(for/or ([x xs]) p)`	`(ormap (lambda (x) p) xs)`
`(for/and ([x xs]) p)`	`(andmap (lambda (x) p) xs)`
`(for/sum ([x xs]) e)`	`(apply + (map (lambda (x) e) xs))`
`(for/fold ([a z]) ([x xs]) e)`	`(foldl (lambda (x a) e) z xs)`
`(for*/list ([x xs] [y ys]) e)`	a `map` inside a `map`, then `apply append`
`(for ([x xs]) e)`	no function: it is run for the effects

The form names the *shape* of the result.
The clauses say *which* elements go in.

Exercise: `splits`

Exercise 4.2

Implement `splits`, which lists every way to cut a list in two.

```
(splits '(a b c))
; '(((a b c) ())
;   ((a b) (c))
;   ((a) (b c))
;   (() (a b c)))
```

Exercise: `splits`

Exercise 4.2

Implement `splits`, which lists every way to cut a list in two.

```
(splits '(a b c))
; '(((a b c) ())
;   ((a b) (c))
;   ((a) (b c))
;   (() (a b c)))
```

Hint

`(take xs 2)` is the first two elements, `(drop xs 2)` is the rest.
A cut is a number between zero and the length.

5 min break



Sequences and Streams

Sequences

A **sequence** is anything a **`for`** form can iterate over:[1]

1. lists, e.g. `'(1 2 3 4)`
2. strings, e.g. `"abcde"`, one character at a time
3. `(in-range 1 5)` and `(in-naturals 1)`
4. a natural number: `4` acts like `(in-range 4)`
5. streams and generators, coming next

[1] See the Racket Reference §4.17.1 for the full list.

Sequences

A **sequence** is anything a **`for`** form can iterate over:[1]

1. lists, e.g. `'(1 2 3 4)`
2. strings, e.g. `"abcde"`, one character at a time
3. `(in-range 1 5)` and `(in-naturals 1)`
4. a natural number: `4` acts like `(in-range 4)`
5. streams and generators, coming next

```
(for/list ([c "abc"]) c)  ; '(#\a #\b #\c)
(for/list ([i 4]) i)      ; '(0 1 2 3)
```

[1] See the Racket Reference §4.17.1 for the full list.

Streams

A **stream** is a lazy list: it produces its elements on demand.

Streams

A **stream** is a lazy list: it produces its elements on demand.

- `empty-stream`, the empty stream
- `(stream-empty? s)` tells whether `s` is empty

Streams

A **stream** is a lazy list: it produces its elements on demand.

- `empty-stream`, the empty stream
- `(stream-empty? s)` tells whether `s` is empty
- `(stream-cons x s)` makes a non-empty stream **without evaluating** `x` or `s`

Streams

A **stream** is a lazy list: it produces its elements on demand.

- `empty-stream`, the empty stream
- `(stream-empty? s)` tells whether `s` is empty
- `(stream-cons x s)` makes a non-empty stream **without evaluating** `x` or `s`
- `(stream-first s)` computes the first element of `s`
- `(stream-rest s)` computes the rest of `s`, as a stream, and nothing inside it

Streams

A **stream** is a lazy list: it produces its elements on demand.

- `empty-stream`, the empty stream
- `(stream-empty? s)` tells whether `s` is empty
- `(stream-cons x s)` makes a non-empty stream **without evaluating** `x` or `s`
- `(stream-first s)` computes the first element of `s`
- `(stream-rest s)` computes the rest of `s`, as a stream, and nothing inside it

Every list function has a stream version, e.g. `stream-map` and `stream->list`.

Streams: A Stream With No End



```
(define (from n)
  (stream-cons n (from (+ n 1))))

(from 0)                                    ; #<stream>
(stream-first (stream-rest (from 0)))       ; 1
(for/list ([n (from 0)] [i 5]) n)           ; '(0 1 2 3 4)
```

Streams: A Stream With No End

```
(define (from n)
  (stream-cons n (from (+ n 1))))

(from 0)                                    ; #<stream>
(stream-first (stream-rest (from 0)))       ; 1
(for/list ([n (from 0)] [i 5]) n)           ; '(0 1 2 3 4)
```

Question

`from` calls itself with no base case. Why does `(from 0)` return at all?

Streams: A Stream With No End

```
(define (from n)
  (stream-cons n (from (+ n 1))))

(from 0)                                    ; #<stream>
(stream-first (stream-rest (from 0)))       ; 1
(for/list ([n (from 0)] [i 5]) n)           ; '(0 1 2 3 4)
```

Question

`from` calls itself with no base case. Why does `(from 0)` return at all?

`stream-cons` does not evaluate its arguments.
The call `(from 1)` runs only when `stream-rest` asks for it, and then it returns for the same reason.

Streams: On Demand, and Once

Consider a function that reports when it runs:

```
(define (loud n)
  (displayln (format "computing ~a" n))
  n)

(define s (stream-cons (loud 1) (stream-cons (loud 2) empty-stream)))
; prints nothing
(stream-first s)                ; prints "computing 1", then 1
(stream-first s)                ; 1, and prints nothing
(stream-first (stream-rest s))  ; prints "computing 2", then 2
```

Streams: On Demand, and Once

Consider a function that reports when it runs:

```
(define (loud n)
  (displayln (format "computing ~a" n))
  n)

(define s (stream-cons (loud 1) (stream-cons (loud 2) empty-stream)))
; prints nothing
(stream-first s)                ; prints "computing 1", then 1
(stream-first s)                ; 1, and prints nothing
(stream-first (stream-rest s))  ; prints "computing 2", then 2
```

Note that nothing is computed at **`define`**.

Each element is computed when first asked for, and remembered after that.

Streams: On Demand, and Once

Consider a function that reports when it runs:

```
(define (loud n)
  (displayln (format "computing ~a" n))
  n)

(define s (stream-cons (loud 1) (stream-cons (loud 2) empty-stream)))
; prints nothing
(stream-first s)                ; prints "computing 1", then 1
(stream-first s)                ; 1, and prints nothing
(stream-first (stream-rest s))  ; prints "computing 2", then 2
```

Note that nothing is computed at **`define`**.
Each element is computed when first asked for, and remembered after that.

So a stream is a *value*: asking twice gives the same answer at no extra cost.

Streams: From a List, on Demand



The students with a grade above `4.0`, one at a time:

```
(define (names-above-4 students)
  (cond
    [(empty? students) empty-stream]
    [(> (cdr (first students)) 4.0)
     (stream-cons (car (first students))
                  (names-above-4 (rest students)))]
    [else (names-above-4 (rest students))]))

(names-above-4 students)                    ; #<stream>
(stream-first (names-above-4 students))     ; "Anna"
(for ([name (names-above-4 students)])
  (displayln name))                         ; Anna, then Charles
```

Streams: From a List, on Demand



The students with a grade above `4.0`, one at a time:

```
(define (names-above-4 students)
  (cond
    [(empty? students) empty-stream]
    [(> (cdr (first students)) 4.0)
     (stream-cons (car (first students))
                  (names-above-4 (rest students)))]
    [else (names-above-4 (rest students))]))

(names-above-4 students)                  ; #<stream>
(stream-first (names-above-4 students))   ; "Anna"
(for ([name (names-above-4 students)])
  (displayln name))                       ; Anna, then Charles
```

This is a recursive `filter`, with **`stream-cons`** in place of `cons`.
The first name is ready before the rest of the list is looked at.

Streams With State: A Running Balance

Consider an account with a starting balance and a sequence of transactions. We want the balance after each transaction, as a stream:

```
(define (balances balance txs)
  (cond [(stream-empty? txs) empty-stream]
        [else (let ([new (+ balance (stream-first txs))])
                (stream-cons new (balances new (stream-rest txs))))]))

(for/list ([b (balances 100 '(20 -50 10))]) b)  ; '(120 70 80)
(for/list ([b (balances 0 (from 1))] [i 5]) b)  ; '(1 3 6 10 15)
```

Streams With State: A Running Balance

Consider an account with a starting balance and a sequence of transactions. We want the balance after each transaction, as a stream:

```
(define (balances balance txs)
  (cond [(stream-empty? txs) empty-stream]
        [else (let ([new (+ balance (stream-first txs))])
                (stream-cons new (balances new (stream-rest txs))))]))

(for/list ([b (balances 100 '(20 -50 10))]) b)  ; '(120 70 80)
(for/list ([b (balances 0 (from 1))] [i 5]) b)  ; '(1 3 6 10 15)
```

The **state** here is the balance, a parameter, so we never update it in place. Each element is the state after one step, and the rest starts from that state.

Streams With State: A Running Balance

Consider an account with a starting balance and a sequence of transactions. We want the balance after each transaction, as a stream:

```
(define (balances balance txs)
  (cond [(stream-empty? txs) empty-stream]
        [else (let ([new (+ balance (stream-first txs))])
                (stream-cons new (balances new (stream-rest txs))))]))

(for/list ([b (balances 100 '(20 -50 10))]) b)  ; '(120 70 80)
(for/list ([b (balances 0 (from 1))] [i 5]) b)  ; '(1 3 6 10 15)
```

The **state** here is the balance, a parameter, so we never update it in place. Each element is the state after one step, and the rest starts from that state.

Note that **`for/fold`** with the same accumulator gives only the final balance. Here, every balance is produced on demand, even for endless transactions.

Exercise: `retry-delays`

Exercise 4.3

A client that fails to reach a server tries again after a delay, doubles the delay each time, and gives up after a fixed number of attempts.
Define `retry-delays`, the stream of the delays it waits.

```
(for/list ([d (retry-delays 1 5)]) d)  ; '(1 2 4 8 16)
(stream-empty? (retry-delays 1 0))     ; #t
```

Exercise: `retry-delays`

Exercise 4.3

A client that fails to reach a server tries again after a delay, doubles the delay each time, and gives up after a fixed number of attempts.
Define `retry-delays`, the stream of the delays it waits.

```
(for/list ([d (retry-delays 1 5)]) d)  ; '(1 2 4 8 16)
(stream-empty? (retry-delays 1 0))     ; #t
```

Hint

`from` carried one number along and never ended.
This stream carries two, the delay and the attempts left, and ends.

Exercise: `retry-delays` (Solution)

```
(define (retry-delays delay attempts)
  (cond [(= attempts 0) empty-stream]
        [else (stream-cons delay
                           (retry-delays (* 2 delay) (- attempts 1)))]))

(for/list ([d (retry-delays 1 5)]) d)  ; '(1 2 4 8 16)
```

Exercise: `retry-delays` (Solution)

```
(define (retry-delays delay attempts)
  (cond [(= attempts 0) empty-stream]
        [else (stream-cons delay
                           (retry-delays (* 2 delay) (- attempts 1)))]))

(for/list ([d (retry-delays 1 5)]) d)  ; '(1 2 4 8 16)
```

This is a *state machine*: its state is the delay and the attempts left.
Each element is the current state, and the rest starts from the next state.

Exercise: `retry-delays` (Solution)

```
(define (retry-delays delay attempts)
  (cond [(= attempts 0) empty-stream]
        [else (stream-cons delay
                           (retry-delays (* 2 delay) (- attempts 1)))]))

(for/list ([d (retry-delays 1 5)]) d)  ; '(1 2 4 8 16)
```

This is a *state machine*: its state is the delay and the attempts left.
Each element is the current state, and the rest starts from the next state.

These are the same two clauses as in a recursive function over a number.
With **`stream-cons`** in place of `cons`, the next state is computed only on demand.

`grep | head`, in Racket

Consider the log as a stream of lines, and the first three errors in it:

```
(define (error? line) (string-prefix? line "ERROR"))

(for/list ([line (stream-filter error? log)]
           [i 3])
  line)
; '("ERROR disk" "ERROR net" "ERROR cpu")
```

Consider the log as a stream of lines, and the first three errors in it:

```
(define (error? line) (string-prefix? line "ERROR"))

(for/list ([line (stream-filter error? log)]
           [i 3])
  line)
; '("ERROR disk" "ERROR net" "ERROR cpu")
```

Here `stream-filter` is the version of `filter` that produces on demand.
The loop stops after three matches, so nothing after the third one is read.

Consider the log as a stream of lines, and the first three errors in it:

```
(define (error? line) (string-prefix? line "ERROR"))

(for/list ([line (stream-filter error? log)]
           [i 3])
  line)
; '("ERROR disk" "ERROR net" "ERROR cpu")
```

Here `stream-filter` is the version of `filter` that produces on demand.
The loop stops after three matches, so nothing after the third one is read.

When `log` never ends, as with `tail -f`, this works for the same reason.

The Test on Problem Set 3: `group`

Recall `group`, which cuts a list into stretches of alike neighbours.

A common solution uses a helper with two accumulators:

```
(define (group same? xs)
  (cond [(empty? xs) '()]
        [else (go same? (rest xs) (list (first xs)) '())]))
(define (go same? xs run runs)      ; run: this stretch, backwards
  (cond [(empty? xs)                ; runs: the finished stretches, backwards
         (reverse (cons (reverse run) runs))]
        [(same? (first run) (first xs))
         (go same? (rest xs) (cons (first xs) run) runs)]
        [else
         (go same? (rest xs) (list (first xs)) (cons (reverse run) runs))]))
```

The Test on Problem Set 3: `group`

Recall `group`, which cuts a list into stretches of alike neighbours.
A common solution uses a helper with two accumulators:

```
(define (group same? xs)
  (cond [(empty? xs) '()]
        [else (go same? (rest xs) (list (first xs)) '())]))
(define (go same? xs run runs)      ; run: this stretch, backwards
  (cond [(empty? xs)                ; runs: the finished stretches, backwards
         (reverse (cons (reverse run) runs))]
        [(same? (first run) (first xs))
         (go same? (rest xs) (cons (first xs) run) runs)]
        [else
         (go same? (rest xs) (list (first xs)) (cons (reverse run) runs))]))
```

This solution is correct. However, it fails the test twice.
Nothing is produced before `xs` is empty, and `reverse` needs the whole list.

`group`, Written for a Stream: Two Helpers

Say what the result *is*: the first stretch, followed by the stretches of the rest.
Write helpers by explicit recursion, which stop at the end of the first stretch:

```
; x, then the front of xs alike it:  (stretch = 1 '(1 2 3)) is '(1 1)
(define (stretch same? x xs)
  (cond [(and (cons? xs) (same? x (first xs)))
         (cons x (stretch same? (first xs) (rest xs)))]
        [else (list x)]))

; what follows that stretch:  (after-stretch = 1 '(1 2 3)) is '(2 3)
(define (after-stretch same? x xs)
  (cond [(and (cons? xs) (same? x (first xs)))
         (after-stretch same? (first xs) (rest xs))]
        [else xs]))
```

`group`, Written for a Stream: Two Helpers

Say what the result *is*: the first stretch, followed by the stretches of the rest.
Write helpers by explicit recursion, which stop at the end of the first stretch:

```
; x, then the front of xs alike it:  (stretch = 1 '(1 2 3)) is '(1 1)
(define (stretch same? x xs)
  (cond [(and (cons? xs) (same? x (first xs)))
         (cons x (stretch same? (first xs) (rest xs)))]
        [else (list x)]))

; what follows that stretch:  (after-stretch = 1 '(1 2 3)) is '(2 3)
(define (after-stretch same? x xs)
  (cond [(and (cons? xs) (same? x (first xs)))
         (after-stretch same? (first xs) (rest xs))]
        [else xs]))
```

Note that neither looks past the first element that is not alike.
So both pass the incremental test.

`group`, Written for a Stream

```
(define (group same? xs)
  (cond [(empty? xs) '()]
        [else (cons (stretch same? (first xs) (rest xs))
                    (group same? (after-stretch same? (first xs) (rest xs))))]))

(group = '(1 1 2 3 3 3))  ; '((1 1) (2) (3 3 3))
```

`group`, Written for a Stream



```
(define (group same? xs)
  (cond [(empty? xs) '()]
        [else (cons (stretch same? (first xs) (rest xs))
                    (group same? (after-stretch same? (first xs) (rest xs))))]))

(group = '(1 1 2 3 3 3))  ; '((1 1) (2) (3 3 3))
```

The first stretch is produced before the rest of the list is looked at.
The list is walked once, without an accumulator and without `reverse`.

`group`, Written for a Stream

```
(define (group same? xs)
  (cond [(empty? xs) '()]
        [else (cons (stretch same? (first xs) (rest xs))
                    (group same? (after-stretch same? (first xs) (rest xs))))]))

(group = '(1 1 2 3 3 3))  ; '((1 1) (2) (3 3 3))
```

The first stretch is produced before the rest of the list is looked at.
The list is walked once, without an accumulator and without `reverse`.

Note that the code now reads as the definition of the result.

`group` on a Stream With No End

The same definition works for a stream, with **`stream-cons`** in place of `cons`:

```
(define (stream-group same? s)
  (cond [(stream-empty? s) empty-stream]
        [else (let ([x (stream-first s)] [xs (stream-rest s)])
                (stream-cons (stream-stretch same? x xs)
                             (stream-group same? (stream-after-stretch same? x xs))))]))

(define (same-tens? a b) (= (quotient a 10) (quotient b 10)))
(for/list ([g (stream-group same-tens? (from 7))] [i 3]) g)
; '((7 8 9)
;   (10 11 12 13 14 15 16 17 18 19)
;   (20 21 22 23 24 25 26 27 28 29))
```

`group` on a Stream With No End

The same definition works for a stream, with **`stream-cons`** in place of `cons`:

```
(define (stream-group same? s)
  (cond [(stream-empty? s) empty-stream]
        [else (let ([x (stream-first s)] [xs (stream-rest s)])
                (stream-cons (stream-stretch same? x xs)
                             (stream-group same? (stream-after-stretch same? x xs))))]))

(define (same-tens? a b) (= (quotient a 10) (quotient b 10)))
(for/list ([g (stream-group same-tens? (from 7))] [i 3]) g)
; '((7 8 9)
;   (10 11 12 13 14 15 16 17 18 19)
;   (20 21 22 23 24 25 26 27 28 29))
```

The two helpers change in the same way.

`group` on a Stream With No End

The same definition works for a stream, with **`stream-cons`** in place of `cons`:

```
(define (stream-group same? s)
  (cond [(stream-empty? s) empty-stream]
        [else (let ([x (stream-first s)] [xs (stream-rest s)])
                (stream-cons (stream-stretch same? x xs)
                             (stream-group same? (stream-after-stretch same? x xs))))]))

(define (same-tens? a b) (= (quotient a 10) (quotient b 10)))
(for/list ([g (stream-group same-tens? (from 7))] [i 3]) g)
; '((7 8 9)
;   (10 11 12 13 14 15 16 17 18 19)
;   (20 21 22 23 24 25 26 27 28 29))
```

The two helpers change in the same way.

The accumulator version has no stream counterpart: it needs the whole input. Thus, the version that passes the test is the one to write for lists as well.

Generators

Generators: `generator` and `yield`

A **generator** is an imperative object that produces values one call at a time.

Generators: `generator` and `yield`

A **generator** is an imperative object that produces values one call at a time.

- A generator is called like a function, with no arguments.
- Each call runs the body until the next `yield`, and returns what is yielded.

Generators: `generator` and `yield`

A **generator** is an imperative object that produces values one call at a time.

- A generator is called like a function, with no arguments.
- Each call runs the body until the next `yield`, and returns what is yielded.
- The next call *resumes* from that point: the generator keeps its state.

Generators: `generator` and `yield`

A **generator** is an imperative object that produces values one call at a time.

- A generator is called like a function, with no arguments.
- Each call runs the body until the next `yield`, and returns what is yielded.
- The next call *resumes* from that point: the generator keeps its state.

To use them:

```
(require racket/generator)
```

Generators: `generator` and `yield`

A **generator** is an imperative object that produces values one call at a time.

- A generator is called like a function, with no arguments.
- Each call runs the body until the next `yield`, and returns what is yielded.
- The next call *resumes* from that point: the generator keeps its state.

To use them:

```
(require racket/generator)
```

- `(generator () body)` wraps `body` into a generator
- `(yield v)` hands out `v` and pauses `body` until the next call

Generators: `generator` and `yield`



A **generator** is an imperative object that produces values one call at a time.

- A generator is called like a function, with no arguments.
- Each call runs the body until the next `yield`, and returns what is yielded.
- The next call *resumes* from that point: the generator keeps its state.

To use them:

```
(require racket/generator)
```

- `(generator () body)` wraps `body` into a generator
- `(yield v)` hands out `v` and pauses `body` until the next call

If you have used **`yield`** in Python, this is essentially the same construct.

Generators (Example)

The names above `4.0`, one per call:

```
(define next-above-4
  (generator ()
    (for ([s students]
          #:when (> (cdr s) 4.0))
      (yield (car s)))
    "no more"))
(next-above-4)  ; "Anna"
(next-above-4)  ; "Charles"
(next-above-4)  ; "no more"
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Generators (Example)

The names above `4.0`, one per call:

```
(define next-above-4
  (generator ()
    (for ([s students]
          #:when (> (cdr s) 4.0))
      (yield (car s)))
    "no more"))
(next-above-4)  ; "Anna"
(next-above-4)  ; "Charles"
(next-above-4)  ; "no more"
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Each call runs the body to the next `yield` and returns what is yielded.

Generators (Example)

The names above `4.0`, one per call:

```
(define next-above-4
  (generator ()
    (for ([s students]
          #:when (> (cdr s) 4.0))
      (yield (car s)))
    "no more"))
(next-above-4)  ; "Anna"
(next-above-4)  ; "Charles"
(next-above-4)  ; "no more"
```

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

Each call runs the body to the next `yield` and returns what is yielded.

Once the body has returned, every further call returns that final value.

Exercise: `next-delay`



Exercise 4.4

Define `make-next-delay`: a generator of the delays of 4.3, one per call.

```
(define next-delay (make-next-delay 1 3))
(next-delay) ; 1
(next-delay) ; 2
(next-delay) ; 4
(next-delay) ; "give up"
```

Exercise: `next-delay`

Exercise 4.4

Define `make-next-delay`: a generator of the delays of 4.3, one per call.

```
(define next-delay (make-next-delay 1 3))
(next-delay) ; 1
(next-delay) ; 2
(next-delay) ; 4
(next-delay) ; "give up"
```

Hint

Write the two clauses of 4.3 again, with `yield` in place of **`stream-cons`**.

Streams and Generators, Side by Side

A stream is a value.

- `stream-first` reads the first element and changes nothing.
- The rest is another stream, `(stream-rest s)`.
- Asking twice gives the same element, computed once.
- Written by recursion, with **`stream-cons`**.

A generator is an object with state.

- Each call moves it on and returns the next element.
- There is no way back to the start, except making a new generator.
- Asking twice gives two different elements.
- Any code with `yield` in it.

Streams and Generators, Side by Side

A stream is a value.

- `stream-first` reads the first element and changes nothing.
- The rest is another stream, `(stream-rest s)`.
- Asking twice gives the same element, computed once.
- Written by recursion, with **`stream-cons`**.

A generator is an object with state.

- Each call moves it on and returns the next element.
- There is no way back to the start, except making a new generator.
- Asking twice gives two different elements.
- Any code with `yield` in it.

Both are sequences, so both can be used in a **`for`** form.

Streams and Generators, Side by Side



A stream is a value.

- `stream-first` reads the first element and changes nothing.
- The rest is another stream, `(stream-rest s)`.
- Asking twice gives the same element, computed once.
- Written by recursion, with **`stream-cons`**.

A generator is an object with state.

- Each call moves it on and returns the next element.
- There is no way back to the start, except making a new generator.
- Asking twice gives two different elements.
- Any code with `yield` in it.

Both are sequences, so both can be used in a **`for`** form.

A stream is the functional solution, and the substitution model still applies.
A generator is the imperative one, since a call has an effect on the next call.

Why Prefer a Stream: Fusion

Consider two stages of a pipeline, and one stage that does the same:

```
(stream-map f (stream-map g s))
(stream-map (lambda (x) (f (g x))) s)
```

[2] Philip Wadler (1990). **"Deforestation: Transforming Programs to Eliminate Trees".** In: *Theoretical Computer Science* 73.2.

[3] Duncan Coutts, Roman Leshchinskiy, and Don Stewart (2007). **"Stream Fusion: From Lists to Streams to Nothing at All".** In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*.

Why Prefer a Stream: Fusion

Consider two stages of a pipeline, and one stage that does the same:

```
(stream-map f (stream-map g s))
(stream-map (lambda (x) (f (g x))) s)
```

These are equal for every `f`, `g` and `s`, by the proof of problem set 3.
Thus, a compiler may replace the first by the second.

[2] Philip Wadler (1990). **"Deforestation: Transforming Programs to Eliminate Trees".** In: *Theoretical Computer Science* 73.2.

[3] Duncan Coutts, Roman Leshchinskiy, and Don Stewart (2007). **"Stream Fusion: From Lists to Streams to Nothing at All".** In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*.

Why Prefer a Stream: Fusion

Consider two stages of a pipeline, and one stage that does the same:

```
(stream-map f (stream-map g s))
(stream-map (lambda (x) (f (g x))) s)
```

These are equal for every `f`, `g` and `s`, by the proof of problem set 3.
Thus, a compiler may replace the first by the second.

This is known as **fusion**: *deforestation* for trees[2], *stream fusion* for streams[3].

[2] Philip Wadler (1990). **"Deforestation: Transforming Programs to Eliminate Trees".** In: *Theoretical Computer Science* 73.2.

[3] Duncan Coutts, Roman Leshchinskiy, and Don Stewart (2007). **"Stream Fusion: From Lists to Streams to Nothing at All".** In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*.

Why Prefer a Stream: Fusion

Consider two stages of a pipeline, and one stage that does the same:

```
(stream-map f (stream-map g s))
(stream-map (lambda (x) (f (g x))) s)
```

These are equal for every `f`, `g` and `s`, by the proof of problem set 3.
Thus, a compiler may replace the first by the second.

This is known as **fusion**: *deforestation* for trees[2], *stream fusion* for streams[3].

Note that two generators **cannot** be fused this way, since each carries state.

[2] Philip Wadler (1990). **"Deforestation: Transforming Programs to Eliminate Trees".** In: *Theoretical Computer Science* 73.2.

[3] Duncan Coutts, Roman Leshchinskiy, and Don Stewart (2007). **"Stream Fusion: From Lists to Streams to Nothing at All".** In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*.

Programmer-Defined Datatypes

Programmer-Defined Datatypes: `struct`

Values in a pipeline should have names for their parts.
A **`struct`** declaration defines a datatype with named fields:

```
(struct student (firstname lastname grade))
```

Programmer-Defined Datatypes: `struct`

Values in a pipeline should have names for their parts.
A **`struct`** declaration defines a datatype with named fields:

```
(struct student (firstname lastname grade))
```

One line defines five functions:

- `student`, the **constructor**: `(student "Anna" "Ivanova" 4.3)`
- `student?`, the **predicate**: is this value a `student`?
- `student-firstname`, `student-lastname`, `student-grade`, the **accessors**

Programmer-Defined Datatypes: `struct`

Values in a pipeline should have names for their parts.
A **`struct`** declaration defines a datatype with named fields:

```
(struct student (firstname lastname grade))
```

One line defines five functions:

- `student`, the **constructor**: `(student "Anna" "Ivanova" 4.3)`
- `student?`, the **predicate**: is this value a `student`?
- `student-firstname`, `student-lastname`, `student-grade`, the **accessors**

Each name is the struct's name, a dash, and the field's name.
Nothing is checked statically, so a wrong field name fails at run time.

Programmer-Defined Datatypes: `struct` (Example)



```
(struct student (firstname lastname grade))

(define students
  (list (student "Anna"    "Ivanova"  4.3)
        (student "Boris"   "Petrov"   3.6)
        (student "Charles" "Xavier"   4.1)))

(for/list ([s students]
           #:when (> (student-grade s) 4.0))
  (student-lastname s))
; '("Ivanova" "Xavier")
```

Programmer-Defined Datatypes: `struct` (Example)

```
(struct student (firstname lastname grade))

(define students
  (list (student "Anna"    "Ivanova"  4.3)
        (student "Boris"   "Petrov"   3.6)
        (student "Charles" "Xavier"   4.1)))

(for/list ([s students]
           #:when (> (student-grade s) 4.0))
  (student-lastname s))
; '("Ivanova" "Xavier")
```

This is the same query as before, with an accessor in place of `cdr`.
Compare `(student-grade s)` with `(cdr s)`: the code now says what it reads.

Programmer-Defined Datatypes: Why Not a List?

A list of three would also do. However, three things go wrong with it:

```
(define anna (student "Anna" "Ivanova" 4.3))

(student? anna)                                ; #t
(student? (list "Anna" "Ivanova" 4.3))         ; #f
(student-grade (list "Anna" "Ivanova" 4.3)) ; error
anna                                           ; #<student>
```

Programmer-Defined Datatypes: Why Not a List?

A list of three would also do. However, three things go wrong with it:

```
(define anna (student "Anna" "Ivanova" 4.3))

(student? anna)                                ; #t
(student? (list "Anna" "Ivanova" 4.3))         ; #f
(student-grade (list "Anna" "Ivanova" 4.3)) ; error
anna                                           ; #<student>
```

1. `student?` is true for values made by `student`, and for *nothing else*.

Programmer-Defined Datatypes: Why Not a List?

A list of three would also do. However, three things go wrong with it:

```
(define anna (student "Anna" "Ivanova" 4.3))

(student? anna)                                ; #t
(student? (list "Anna" "Ivanova" 4.3))         ; #f
(student-grade (list "Anna" "Ivanova" 4.3)) ; error
anna                                           ; #<student>
```

1. `student?` is true for values made by `student`, and for *nothing else*.
2. An accessor applied to a wrong value is an error, not a wrong answer.

Programmer-Defined Datatypes: Why Not a List?

A list of three would also do. However, three things go wrong with it:

```
(define anna (student "Anna" "Ivanova" 4.3))

(student? anna)                                ; #t
(student? (list "Anna" "Ivanova" 4.3))         ; #f
(student-grade (list "Anna" "Ivanova" 4.3)) ; error
anna                                           ; #<student>
```

1. `student?` is true for values made by `student`, and for *nothing else*.
2. An accessor applied to a wrong value is an error, not a wrong answer.
3. A struct is *opaque*: it prints as `#<student>`, without its fields.
   With **`#:transparent`** after the fields, it prints and compares them.

Programmer-Defined Datatypes: Copy and Update

A struct is immutable.
To change a field, make a copy with that field replaced:

```
(define anna (student "Anna" "Ivanova" 4.0))

(define new-anna (struct-copy student anna [grade 4.5]))

(student-grade new-anna)  ; 4.5
(student-grade anna)      ; 4.0
```

Programmer-Defined Datatypes: Copy and Update

A struct is immutable.
To change a field, make a copy with that field replaced:

```
(define anna (student "Anna" "Ivanova" 4.0))

(define new-anna (struct-copy student anna [grade 4.5]))

(student-grade new-anna)  ; 4.5
(student-grade anna)      ; 4.0
```

Note that `anna` is unchanged.
`struct-copy` takes the struct name, then the value, then the fields to change.

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
2. Which of `filter` and `reverse` would still work on an input with no end?
3. Why does `(from 0)` return, although `from` has no base case?
4. Why is `group` with an accumulator and a `reverse` not written for a stream?
5. A generator is called twice. Does it return the same value?
6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
3. Why does `(from 0)` return, although `from` has no base case?
4. Why is `group` with an accumulator and a `reverse` not written for a stream?
5. A generator is called twice. Does it return the same value?
6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
   `filter`, since its first result needs only the first inputs.
3. Why does `(from 0)` return, although `from` has no base case?

4. Why is `group` with an accumulator and a `reverse` not written for a stream?

5. A generator is called twice. Does it return the same value?

6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
   `filter`, since its first result needs only the first inputs.
3. Why does `(from 0)` return, although `from` has no base case?
   `stream-cons` does not evaluate the rest until it is asked for.
4. Why is `group` with an accumulator and a `reverse` not written for a stream?

5. A generator is called twice. Does it return the same value?

6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
   `filter`, since its first result needs only the first inputs.
3. Why does `(from 0)` return, although `from` has no base case?
   `stream-cons` does not evaluate the rest until it is asked for.
4. Why is `group` with an accumulator and a `reverse` not written for a stream?
   Nothing comes before the end, so produce the first stretch, then the rest.
5. A generator is called twice. Does it return the same value?

6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
   `filter`, since its first result needs only the first inputs.
3. Why does `(from 0)` return, although `from` has no base case?
   `stream-cons` does not evaluate the rest until it is asked for.
4. Why is `group` with an accumulator and a `reverse` not written for a stream?
   Nothing comes before the end, so produce the first stretch, then the rest.
5. A generator is called twice. Does it return the same value?
   No, it returns the next one, since a generator has state.
6. What do the four functions defined by `(struct point (x y))` do?

Quick recap

1. Which **`for`** form replaces `map`, and which replaces `filter`?
   `for/list`, and **`for/list`** with a **`#:when`** clause.
2. Which of `filter` and `reverse` would still work on an input with no end?
   `filter`, since its first result needs only the first inputs.
3. Why does `(from 0)` return, although `from` has no base case?
   `stream-cons` does not evaluate the rest until it is asked for.
4. Why is `group` with an accumulator and a `reverse` not written for a stream?
   Nothing comes before the end, so produce the first stretch, then the rest.
5. A generator is called twice. Does it return the same value?
   No, it returns the next one, since a generator has state.
6. What do the four functions defined by `(`**`struct`**` point (x y))` do?
   `point` makes one, `point?` tests one, `point-x` and `point-y` read a field.

Quiz for Lecture 4

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 4

Ten minutes, on Moodle, now.
Seven questions on this lecture, then three about the lecture itself.

No AI. The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

Say what you learned and what stayed unclear, **specifically**.
I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend problem set 3, if your name is drawn
- start on problem set 4

→ **Before next week: closures, Python, JavaScript**

- Read SICP §3.5.1 (Abelson and Sussman 1996), streams are delayed lists.
- Read the Racket Reference §4.17, sequences, streams and generators, at `https://docs.racket-lang.org/reference/`.
- Finish exercises 4.2 to 4.4, and check each on the example from its slide.

Thank you!



References

- Abelson, Harold and Gerald Jay Sussman (1996). ***Structure and interpretation of computer programs***. The MIT Press.
- Coutts, Duncan, Roman Leshchinskiy, and Don Stewart (2007). **"Stream Fusion: From Lists to Streams to Nothing at All".** In: *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*, pp. 315–326. DOI: 10.1145/1291151.1291199.
- Wadler, Philip (1990). **"Deforestation: Transforming Programs to Eliminate Trees".** In: *Theoretical Computer Science* 73.2, pp. 231–248. DOI: 10.1016/0304-3975(90)90147-A.